

GUI Based Multi-Client Chat Application Using TCP

WITH PRIVATE MESSAGING, CHAT HISTORY, PERFORMANCE ANALYSIS

Submitted By

Name: Kunal Prajapati

Enrollment No.: UU2409000396

Submitted To

ISEA Summer Internship 2026

ISEA under MeitY

(In affiliation with Tezpur University, Assam)

Course Assignment

Assignment 6

GUI Based Multi-Client Chat Application Using TCP

1. OBJECTIVE

The objective of this assignment was to convert the terminal-based Multi-Client TCP Chat Application developed in Assignment 5 into a graphical desktop application while reusing the existing networking logic. The GUI was developed using Python Tkinter to provide an interactive and user-friendly chatting environment.

The application was executed inside a Mininet virtual network consisting of one server and four clients. TCP sockets were used for reliable communication, while multithreading ensured that incoming messages were received without freezing the graphical interface.

The major objectives achieved are:

- Design a GUI-based login window.
- Develop a responsive chat interface.
- Reuse the TCP server implementation from Assignment 5.
- Display connected users dynamically.
- Support broadcast messaging.
- Support private messaging.

- Display join and leave notifications.
 - Maintain GUI responsiveness using background threads.
 - Verify communication using Wireshark.
-

2. GUI DESIGN

The application consists of two major graphical windows.

Login Window

The login window allows the user to:

- Enter Username
- Enter Server IP Address
- Connect to Server
- Exit Application

The window validates user input before establishing a TCP connection.



Main Chat Window

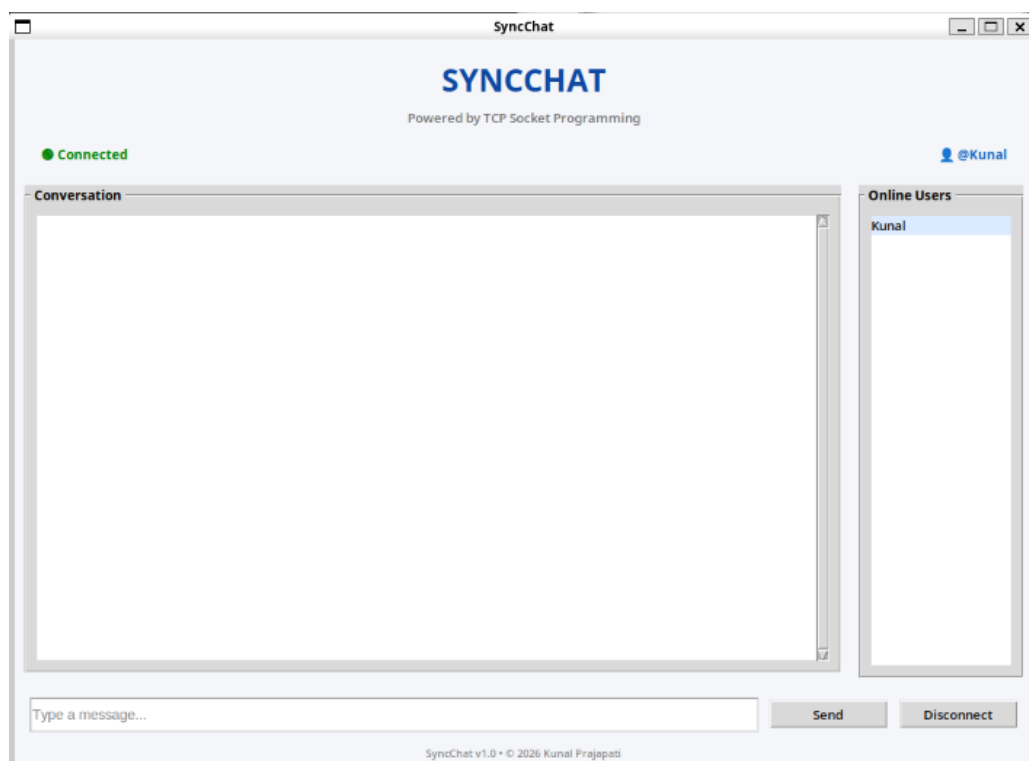
After successful authentication, the chat window opens automatically.

The interface contains:

- Application Header (SyncChat)

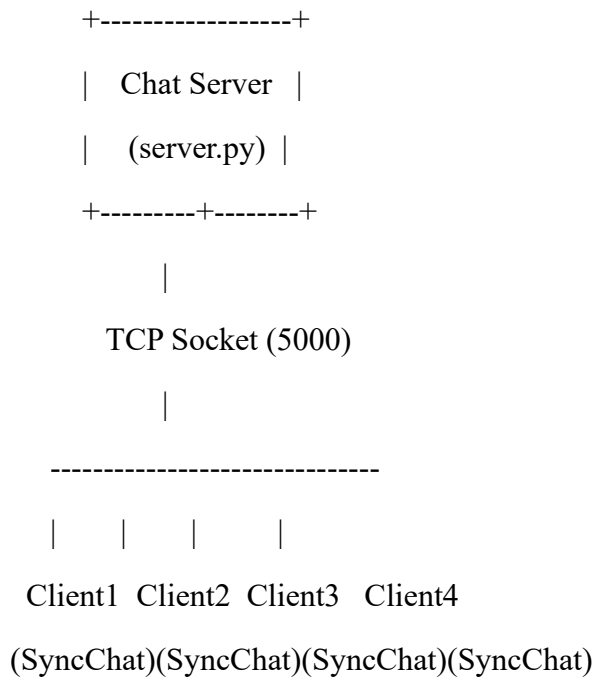
- Connection Status Indicator
- Logged-in User Information
- Scrollable Conversation Area
- Online Users Panel
- Message Input Box
- Send Button
- Disconnect Button
- Footer containing application version and copyright

The interface automatically updates messages and online users without blocking user interaction.



3. SYSTEM ARCHITECTURE

The application follows a Client–Server architecture.



The server performs the following responsibilities:

- Accept multiple client connections
- Maintain connected users
- Broadcast public messages
- Route private messages
- Maintain online user list
- Handle client disconnections
- Send updates to every connected GUI client

The GUI client performs:

- Server connection
- Username registration
- Sending public messages
- Sending private messages
- Displaying online users
- Displaying chat history
- Receiving incoming messages

- Maintaining responsive interface
-

4. COMPONENTS REUSED FROM ASSIGNMENT 5

Instead of redesigning the complete application, the networking layer from Assignment 5 was reused.

The following components were directly reused:

- TCP Socket Communication
- Server Implementation
- Multi-threaded Client Handling
- Broadcast Messaging
- Private Messaging
- Dynamic Online User List
- Join Notifications
- Leave Notifications
- Server Statistics
- Command Processing
- Message Routing

Only the client interface was replaced with a graphical desktop application developed using Tkinter.

This modular design minimized code duplication and maintained compatibility with the existing server.

5. GUI DESIGN DECISIONS

Several improvements were made to enhance usability.

Login Window

- Simple and clean layout
- Default Server IP Address
- Input validation
- Immediate connection feedback

Chat Window

The application branding was changed from:

Advanced Multi Client TCP Chat

to

SyncChat

to provide a modern and recognizable interface.

Additional improvements include:

- Connection status indicator
- Logged-in username display
- Scrollable conversation area
- Online users panel
- Automatic scrolling
- Placeholder text inside message box
- Footer with application version
- Color-coded messages

Message colors were designed as follows:

Message Type Color

Broadcast Blue

Private Purple

System Green

Message Text Black

The currently logged-in user always appears at the top of the Online Users panel.

When displaying messages sent by the current user, the sender name is replaced with **You**, improving readability.

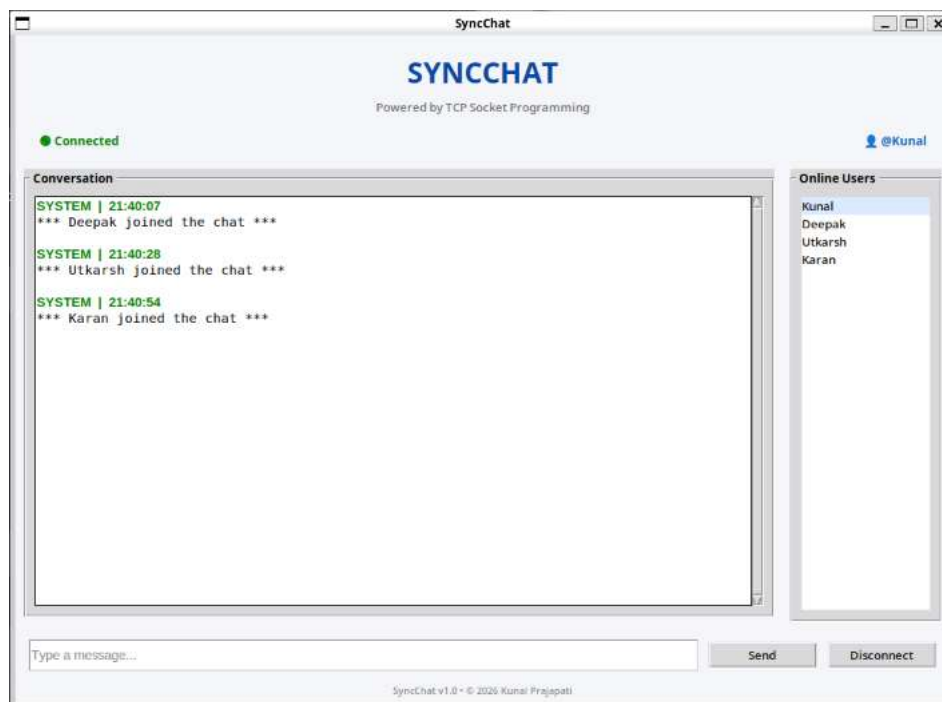
6. TESTING RESULTS

The application was tested inside Mininet using one server and four clients.

The following features were verified successfully.

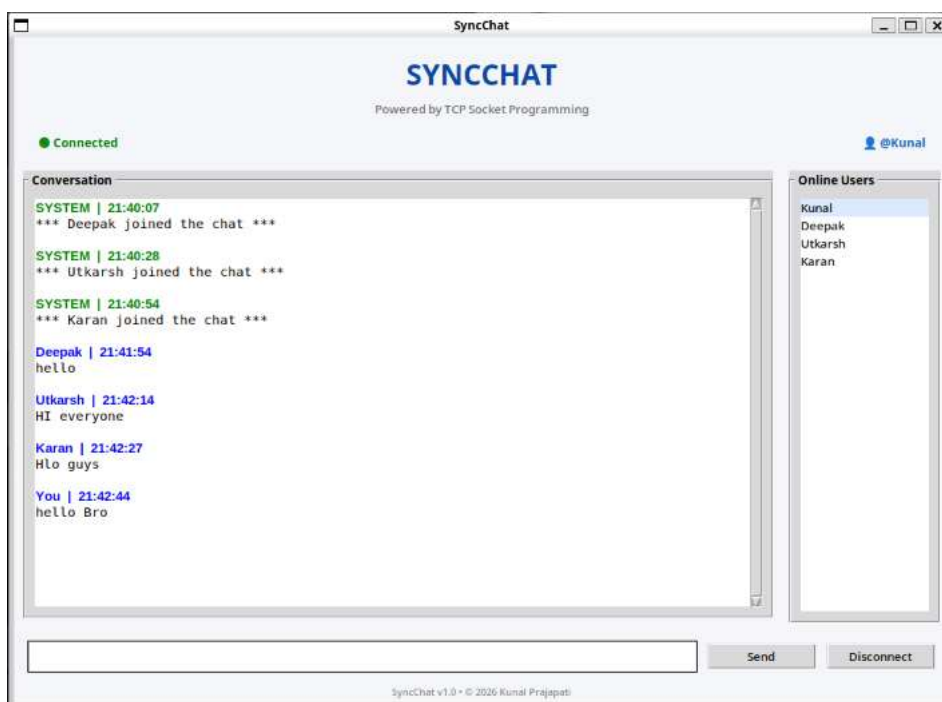
1. Login Verification

Users connected successfully after entering valid usernames.



2. Broadcast Messaging

Messages sent by one client were received by all connected users.

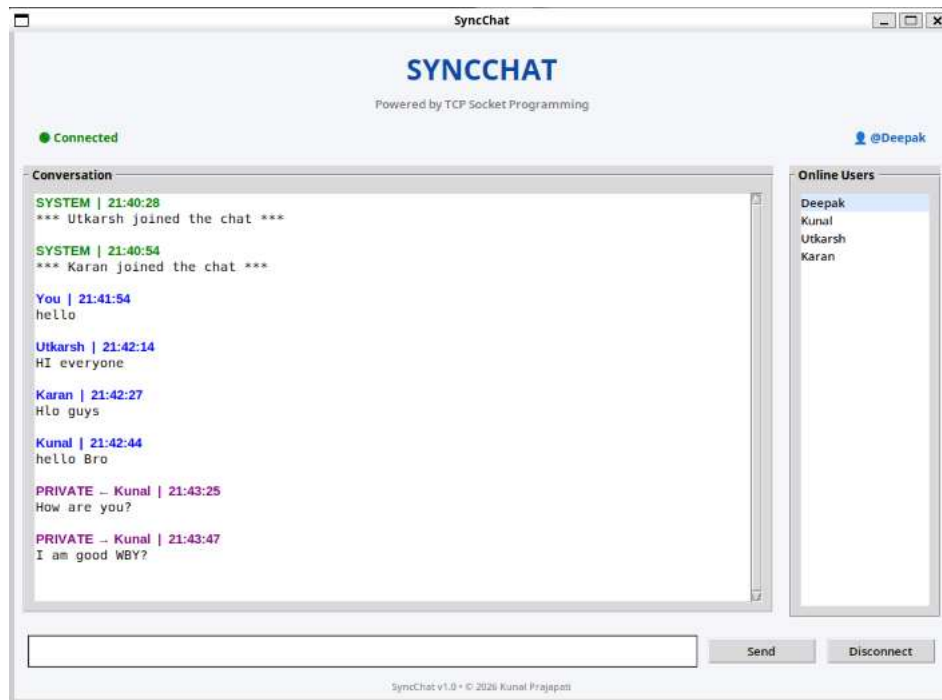


3. Private Messaging

Private communication using:

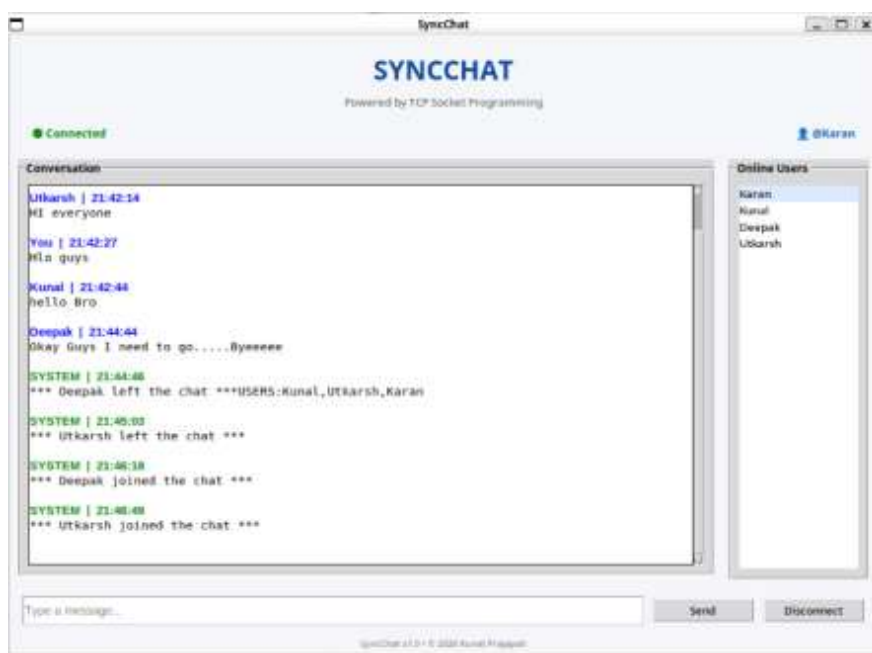
/msg username message

was delivered only to the intended recipient.



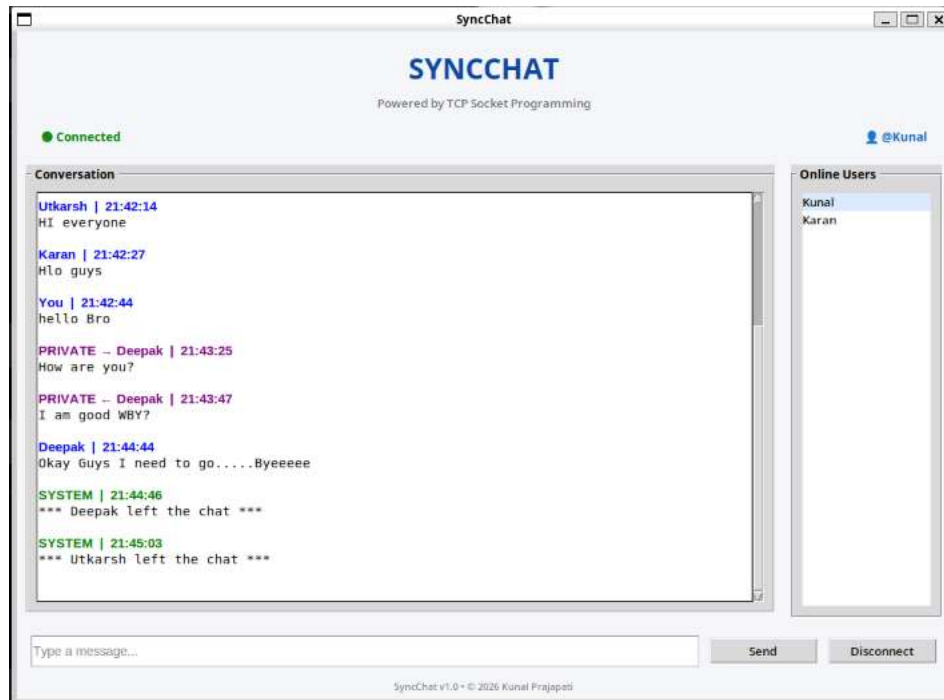
4. User Joining

Whenever a new client joined the server, every connected client received a system notification.



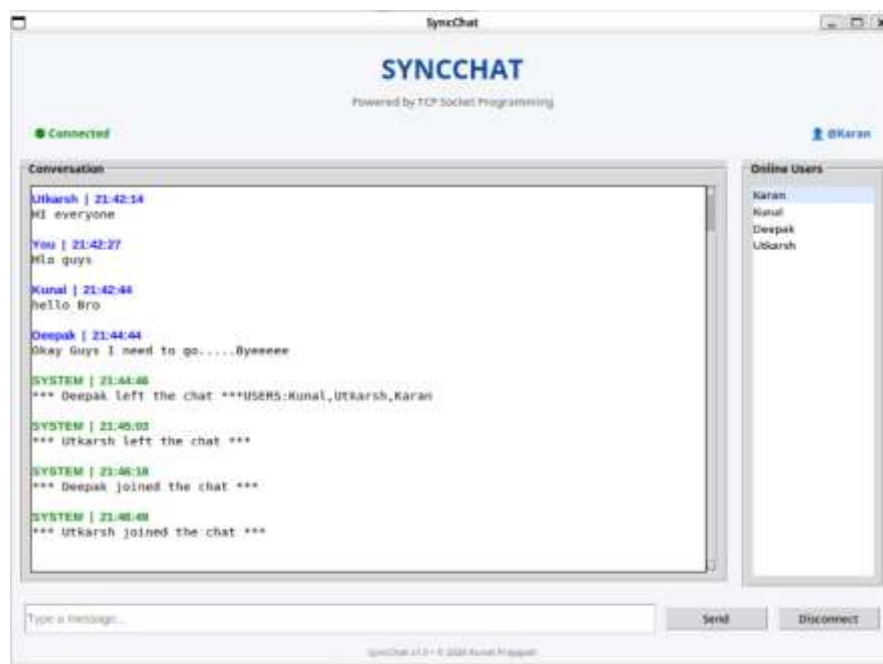
5. User Leaving

Whenever a client disconnected, the server broadcasted a leave notification and updated the online users list automatically.



6. Online Users

The Online Users panel updated dynamically whenever users joined or left.



The currently logged-in user remained highlighted at the top of the list.

7. WIRESHARK VERIFICATION

Wireshark was used to verify TCP communication.

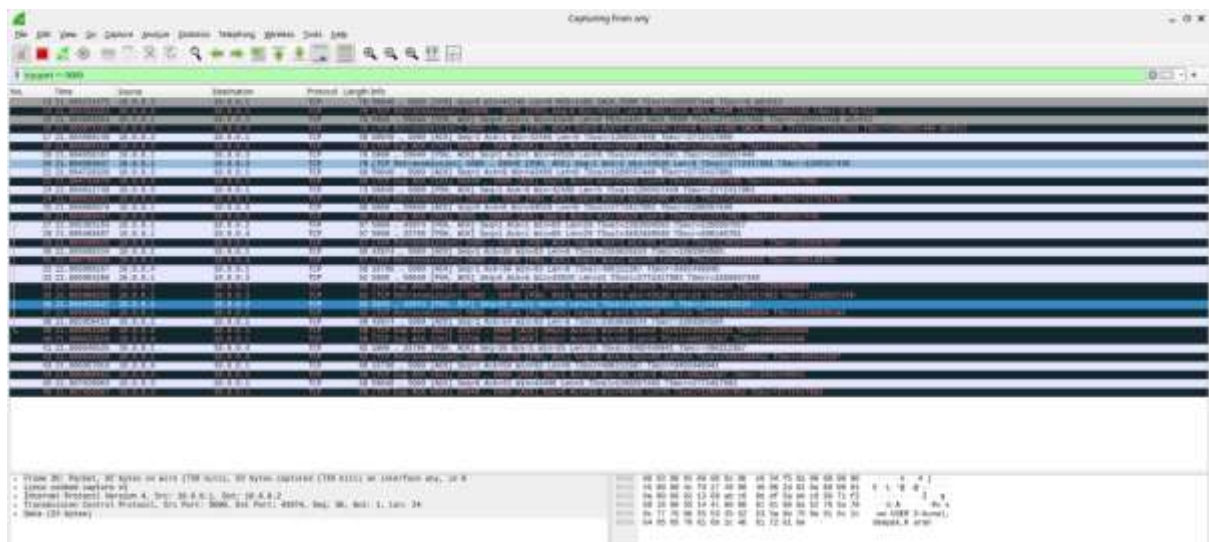
Filter used:

tcp.port == 5000

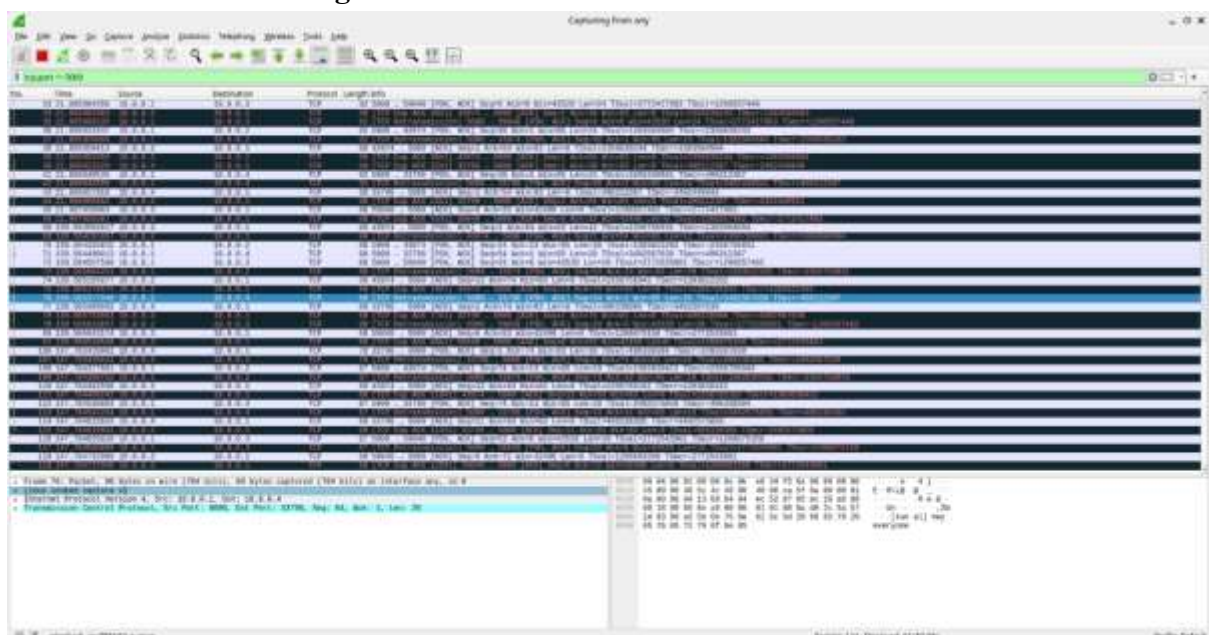
The following packet types were observed.

1. TCP Connection Establishment

The initial TCP three-way handshake (SYN → SYN-ACK → ACK) confirmed successful connection establishment between client and server.



2. Broadcast Message Packet



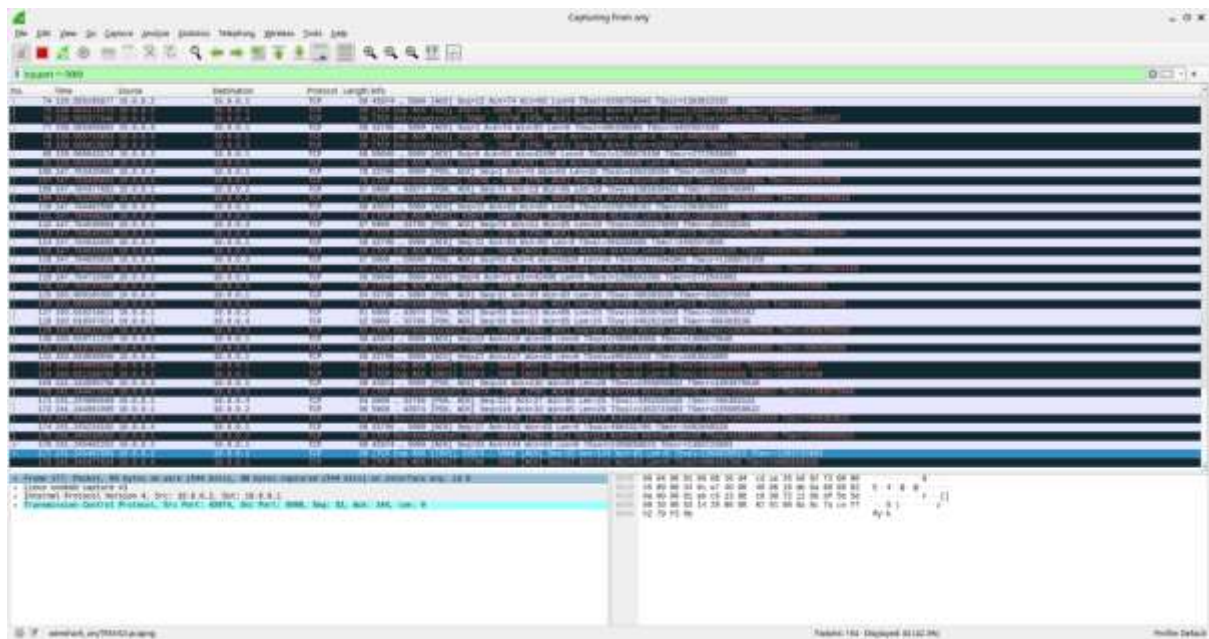
Broadcast communication was verified using TCP PSH-ACK packets carrying chat messages from the server to all connected clients.

The packet payload displayed the transmitted chat message, confirming reliable delivery

Private Message Packet

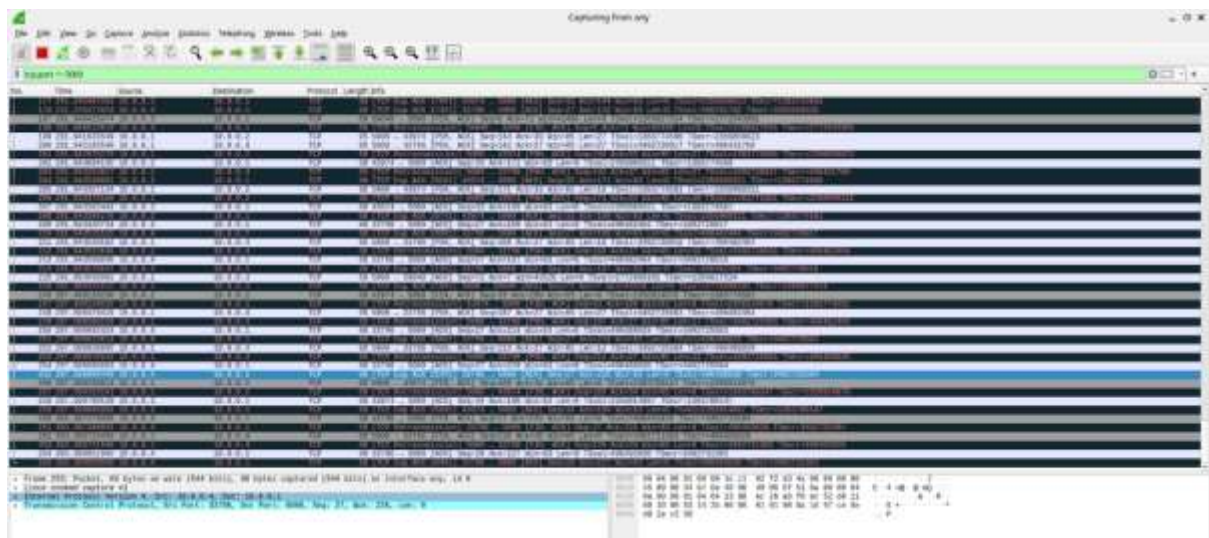
Private messaging packets were captured and verified.

Only the intended recipient received the packet, confirming correct server-side routing.



Client Disconnection

TCP FIN and ACK packets confirmed graceful termination of the client connection.



8. REFLECTION ANSWERS

Q1. Why should networking logic and GUI code be separated?

Separating networking logic from GUI code improves modularity, maintainability, and scalability. The networking module handles socket communication independently, while the GUI focuses on user interaction. This separation simplifies debugging, allows independent testing, and enables future upgrades without affecting the communication layer.

Q2. Why is a background thread required?

Socket operations such as `recv()` are blocking by nature. If executed in the main GUI thread, the interface would freeze while waiting for incoming messages. Running the receive function inside a background thread allows the application to continue responding to user input while simultaneously processing incoming network data.

Q3. Which components from Assignment 5 were reused?

The following components were reused without major modification:

- TCP socket communication
- Multi-threaded server
- Broadcast messaging
- Private messaging
- Online user management
- Join and leave notifications
- Message routing
- Command processing
- Server statistics

Only the terminal-based client interface was replaced with a Tkinter-based graphical interface.

Q4. How did the GUI improve usability?

The graphical interface significantly improved the user experience by replacing terminal commands with interactive controls. Users can view messages in a scrollable chat area,

monitor connected users, send messages through buttons or the Enter key, and receive real-time updates without interacting with the command line.

Q5. Suggest one future enhancement.

Future enhancements may include:

- Secure user authentication
 - Password-protected accounts
 - File sharing support
 - Image transfer
 - Emoji support
 - Dark mode
 - End-to-end encryption
 - Group chat functionality
 - Database-based chat history
 - Voice and video communication
-

9. CONCLUSION

This assignment successfully transformed the terminal-based TCP chat client into a fully functional graphical desktop application while preserving the networking implementation developed in Assignment 5. The resulting application, **SyncChat**, provides a clean and intuitive interface for real-time communication over TCP sockets.

The application successfully supports multiple concurrent clients, public and private messaging, dynamic online user management, connection status monitoring, and graceful disconnection. By employing multithreading, the graphical interface remains responsive even while receiving continuous network traffic.

Testing within the Mininet environment confirmed reliable communication among multiple clients, while Wireshark packet analysis verified TCP connection establishment, message transmission, private message routing, and connection termination. The project demonstrates the practical integration of socket programming, GUI development, concurrent programming, and network analysis, providing valuable experience in designing user-friendly network applications.

Software Used

- Python 3
- Tkinter
- Socket Programming
- Threading
- Mininet
- Wireshark
- Ubuntu (WSL)
- Visual Studio Code